



Plataforma Backend de Temática Pokémon

Memoria Técnica del Proyecto — MVP con PvE
Node.js · Express · Sequelize ORM · PostgreSQL · MongoDB · Docker

Autor	Módulo responsable
Asier Gonzalez	User · Auth · Relaciones · PvE - Carrito
Darío Arenaza	Pokemon · Documentación técnica
Luis Alonso	Product · Vistas (Pug)
Eli Fernández	Team · TeamPokemon · Diagramas

Desarrollo Backend · Mayo 2026

1. Introducción

POKEback es el backend de una plataforma de temática Pokémon desarrollada como proyecto grupal. Su propósito es ofrecer una API REST completa que gestione usuarios, Pokémon, productos de tienda, equipos y un sistema de combate PvE por turnos, sentando las bases para futuras mecánicas PvP.

El proyecto nace con la filosofía de:

"Menos es más, si está bien hecho."

Esto implica priorizar la calidad del código, la coherencia del diseño y la seguridad frente a la cantidad de funcionalidades. Cada decisión técnica ha sido tomada buscando la claridad, la mantenibilidad y las buenas prácticas de la industria.

1.1 Contexto y motivación

El proyecto surge como ejercicio integrador del bootcamp de desarrollo backend, donde el equipo debía demostrar dominio de:

- Diseño e implementación de APIs REST con Express
- Modelado de base de datos relacional con Sequelize y PostgreSQL
- Autenticación segura mediante JWT y gestión de sesiones
- Containerización con Docker y Docker Compose
- Trabajo en equipo con división clara de responsabilidades
- Estructuración de proyectos siguiendo el patrón MVC con capa de servicios
- Diseño e implementación de mecánicas de juego PvE sobre la arquitectura base

2. Tecnologías y Stack Técnico

Tecnología	Rol en el proyecto	Justificación
Node.js v24	Runtime de servidor	Ecosistema JS unificado, alto rendimiento I/O asíncrono
Express.js	Framework web	Minimalista, flexible, amplio soporte de middlewares
Sequelize ORM	Capa de acceso a datos	Abstracción SQL, asociaciones declarativas
PostgreSQL 18	Base de datos relacional	Robustez, relaciones complejas, transacciones ACID
MongoDB 7	Base de datos documental	Disponible para logs y eventos futuros
Docker + Compose	Containerización	Entorno reproducible: postgres, mongo, pgadmin
JSON Web Token	Autenticación sin estado	Escalabilidad, sin sesiones en servidor, estándar
Express-session	Gestión de sesiones web	Complementa JWT para vistas renderizadas
Pug (Jade)	Motor de plantillas	Integración nativa con Express, sintaxis concisa
bcrypt	Hash de contraseñas	Algoritmo estándar con sal para almacenamiento seguro
dotenv	Variables de entorno	Separación de configuración y código fuente

2.1 Decisiones técnicas sobre el stack

¿Por qué Docker Compose?

El proyecto arranca con un único comando (`docker compose up`) levantando tres servicios: PostgreSQL 18, MongoDB 7 y pgAdmin 4. El `init.sql` se monta automáticamente como volumen y ejecuta la inicialización completa de tablas y datos de ejemplo al primer arranque. Esto elimina pasos manuales y garantiza reproducibilidad entre entornos.

¿Por qué JWT + sesiones en paralelo?

La aplicación tiene dos superficies: una API REST (consumida por clientes externos o Postman) y vistas renderizadas en servidor (Pug). JWT resulta ideal para la API por ser stateless. Las sesiones de Express complementan las vistas HTML donde el mantenimiento del estado entre peticiones es necesario. Ambos middlewares coexisten sin interferir al aplicarse selectivamente en cada router.

3. Arquitectura del Sistema

El backend sigue el patrón MVC con una capa de servicios intermedia:

Capa	Responsabilidad	Archivos clave
Rutas (Routes)	Definir endpoints y aplicar middlewares	routes/*.js
Controladores	Recibir la petición HTTP y devolver la respuesta	controllers/*.js
Servicios	Lógica de negocio, validaciones, operaciones complejas	services/*.js
Modelos	Definición del esquema y queries a la base de datos	models/*.js
Middlewares	Autenticación JWT, control de sesión y roles	middlewares/*.js
Vistas	Plantillas Pug renderizadas en servidor	views/*.pug

3.1 Estructura de directorios actual

```
src/
├── config/
│   ├── postgres.js      # Conexión Sequelize + PostgreSQL
│   └── mongo.js         # Conexión MongoDB
├── controllers/
│   ├── battleController.js # PvE: selección, turno, resultado
│   ├── cartController.js
│   └── ...
├── models/
│   ├── battleModel.js    # Historial de batallas
│   ├── orderModel.js + orderItemModel.js
│   └── ...
├── routes/
│   ├── viewRoutes.js     # Todas las vistas SSR incl. /battle
│   └── ...
├── services/
│   ├── battleService.js  # Motor de combate PvE
│   └── ...
└── views/
    ├── battle-select.pug
    ├── battle-fight.pug
    └── battle-result.pug
```

3.2 Decisión: separar Servicios de Controladores

Una decisión clave fue introducir la capa de servicios en lugar de poner la lógica de negocio directamente en los controladores. Esto permite reutilización (un servicio puede ser llamado desde distintos controladores), testabilidad y claridad: los controladores quedan como adaptadores finos entre HTTP y negocio.

4. Modelo de Datos y Relaciones

Relación	Tipo	Significado
User → UserPokemon	One-to-Many	Un usuario puede tener múltiples Pokémon capturados
UserPokemon → Pokemon	Many-to-One	Cada instancia referencia al Pokémon base del catálogo
User → Team	One-to-Many	Un usuario puede tener múltiples equipos
Team → TeamPokemon	One-to-Many	Un equipo tiene hasta 6 slots
TeamPokemon → UserPokemon	Many-to-One	El slot referencia el Pokémon concreto del usuario
User → Order	One-to-Many	Historial de compras del usuario
Order → OrderItem	One-to-Many	Cada orden tiene líneas de producto
User ↔ Product	Many-to-Many	A través de UserStore (carrito/inventario)
Battle → User	Many-to-One	Registro de batallas por usuario
Battle → UserPokemon	Many-to-One (SET NULL)	FK nullable: el Pokémon puede morir tras la batalla
Battle → Pokemon	Many-to-One	Referencia al Pokémon enemigo del catálogo

4.1 Tabla battles — decisión de diseño FK nullable

La FK battles.user_pokemon_id referencia a user_pokemon con ON DELETE SET NULL. Esto permite que el historial de batallas se conserve incluso cuando el Pokémon del jugador muere y es eliminado de user_pokemon. Sin esta decisión, el DELETE del Pokémon fallaba por violación de integridad referencial.

4.2 Catálogo Pokemon vs instancias UserPokemon

El modelo Pokemon actúa como catálogo de datos base (tipo, sprite, evolución). La tabla user_pokemon representa la instancia concreta capturada por un usuario, con stats propios (current_hp, current_attack, current_speed). Esta separación sigue el principio de normalización y permite que los stats evolucionen de forma independiente por usuario.

5. Sistema PvE — Combate por Turnos

El sistema PvE es la funcionalidad de juego principal del MVP. Permite a los usuarios enfrentarse a Pokémon salvajes generados aleatoriamente desde el catálogo, tomar decisiones por turno y ver consecuencias permanentes en su colección.

5.1 Flujo de batalla

Ruta	Método	Descripción
/battle	GET	Selección del Pokémon del usuario (solo muestra los vivos)
/battle/start	POST	Inicia la batalla: genera enemigo aleatorio, crea estado en sesión
/battle/fight	GET	Renderiza el estado actual del combate
/battle/turn	POST	Procesa la acción del turno (attack/capture/flee)
/battle/result	GET	Muestra el resultado final y limpia el estado de sesión

5.2 Acciones por turno

Acción	Efecto jugador	Efecto enemigo
 Atacar	Resta userAttack al HP del enemigo	Contraataca restando enemyAttack al HP del jugador
 Capturar	Intenta captura con % según HP enemigo	Si falla, contraataca igualmente
 Escapar	Fin de batalla sin recompensa	Sin efecto

5.3 Mecánica de captura

Estado del enemigo	Probabilidad de éxito
HP > 50% del máximo	35%
HP ≤ 50% del máximo	65%

Esto crea la decisión estratégica central: debilitar al enemigo aumenta la probabilidad de captura, pero cada ataque recibido acerca la derrota permanente del Pokémon propio.

5.4 Consecuencias permanentes

- Victoria por derrota: +20 monedas al usuario
- Victoria por captura: Pokémon añadido a user_pokemon con HP regenerado al base + +20 monedas
- Derrota: DELETE en user_pokemon — el Pokémon muere para siempre (permadeath)
- Escape: sin recompensa ni penalización

5.5 Escalado del enemigo

Los stats del enemigo se calculan dinámicamente según su tipo, creando variedad sin necesidad de tablas adicionales:

- Tipos agresivos (fire, electric): HP reducido al 90% del base, Ataque al 150%
- Tipos defensivos (resto): HP al 130% del base, Ataque al 80%

Ver el tipo del enemigo antes de atacar es información útil para decidir cuándo intentar la captura.

5.6 Estado de batalla en sesión

El estado completo del combate se almacena en req.session.battle entre turnos, evitando la necesidad de una tabla de batallas activas en la BD. Solo se persiste en la tabla battles al finalizar (win/loss/flee/captured).

6. API REST y Autenticación

6.1 Autenticación y autorización

El sistema implementa dos niveles de protección que coexisten sin interferir:

- JWT (authMiddleware.js): para endpoints de la API REST. El token se envía en Authorization: Bearer <token>.
- Express-session (sessionMiddleware.js): para las vistas renderizadas. Comprueba req.session.user y el campo rol del usuario.

Método	Ruta	Descripción	Auth
POST	/auth/register	Registro con bcrypt	Pública
POST	/auth/login	Login → JWT + sesión	Pública
GET	/api/cart	Ver carrito	JWT
POST	/api/cart/add	Añadir producto	JWT
DELETE	/api/cart/remove/:id	Eliminar producto	JWT
POST	/api/cart/checkout	Finalizar compra	JWT
GET	/api/users	Listar usuarios	Admin JWT
GET	/api/pokemon	Listar catálogo	Pública
POST	/api/pokemon	Crear Pokémon	Admin JWT

GET	/api/products	Listar productos	Pública
POST	/api/products	Crear producto	Admin JWT

7. Retos Técnicos y Soluciones Adoptadas

Reto 1 — Coexistencia de JWT y sesiones

Problema: proteger endpoints REST (sin estado, JWT) y rutas de vistas HTML (donde el navegador no gestiona tokens).

Solución: dos middlewares independientes aplicados selectivamente. `authMiddleware.js` intercepta el header `Authorization`. `sessionMiddleware.js` comprueba `req.session.user`. Las rutas de vistas viven en `viewRoutes.js` con `requireSession`; las rutas API usan `verifyToken`.

Reto 2 — Rutas de vistas vs rutas API

Problema: el router raíz montaba `userStoreRoutes` en `'/'` lo que capturaba todas las rutas posteriores, incluyendo `/battle`. Las rutas de vista del carrito y de batalla no llegaban a ejecutarse.

Solución: centralizar todas las rutas de vistas en `viewRoutes.js` (montado en `'/'`) y asegurarse de que este router sea siempre el último en registrarse en `index.js`. Las rutas API se montan con prefijos explícitos (`/api/cart`, `/api/cart...`) antes del comodín.

Reto 3 — FK nullable en battles para permadeath

Problema: al derrotar al Pokémon del jugador, el flujo requería borrar su registro en `user_pokemon`. Sin embargo, la tabla `battles` tenía una FK NOT NULL apuntando a ese registro, lo que generaba una violación de integridad referencial al intentar el DELETE.

Solución: definir la FK `battles.user_pokemon_id` con `ON DELETE SET NULL` y `allowNull: true` en el modelo Sequelize. El historial de batallas se conserva; el campo queda en NULL cuando el Pokémon muere. El orden también importa: `saveBattle()` debe ejecutarse antes de `killUserPokemon()` para que el registro exista al crearse.

Reto 4 — Estado del combate entre turnos

Problema: cada turno es una petición HTTP independiente, por lo que los datos del combate (HP actuales, stats del enemigo) no persisten automáticamente.

Solución: almacenar el objeto `battle` completo en `req.session.battle`. Al finalizar la batalla (cualquier resultado), se mueve a `req.session.battleResult` para renderizar la pantalla de resultado y se limpia el estado de combate. Esto evita una tabla de batallas activas en BD.

Reto 5 — Sincronización del esquema con Docker

Problema: el `init.sql` se ejecuta solo en el primer arranque del contenedor. Cambios en el esquema requerían borrar el volumen manualmente.

Solución: para desarrollo, `docker compose down -v` elimina los volúmenes y permite re-inicializar limpio. Para el entorno de producción se recomienda migrar a `sequelize-cli` con migraciones versionadas.

Reto 6 — Offset aleatorio con pocos registros en BD

Problema: el servicio de batalla usaba ORDER BY con OFFSET aleatorio para seleccionar un enemigo. Con pocos Pokémon en la BD y un offset fuera de rango, findOne() devolvía null.

Solución: calcular el offset como `Math.floor(Math.random() * (count - 1))` asegurando que siempre esté dentro del rango disponible. El catálogo de 151 Pokémon en el seed garantiza suficiente variedad.

8. Flujo de Datos — Ejemplo: Turno de Batalla

A continuación se detalla el ciclo de vida completo de un turno de combate PvE:

1. El usuario envía POST /battle/turn con body { action: 'attack' }
2. requireSession verifica req.session.user; si no existe redirige a /login
3. battleController.postBattleTurn lee req.session.battle y llama a battleService.processTurn(battle, action)
4. battleService calcula el daño, actualiza HP en el objeto battle, comprueba condiciones de fin
5. Si result !== null: saveBattle() persiste en BD → addCoins() o killUserPokemon() según el resultado
6. El controlador actualiza req.session.battle con el nuevo estado y redirige a /battle/fight (o /battle/result si terminó)
7. battle-fight.pug renderiza las barras de HP actualizadas y las opciones de acción disponibles

Este flujo demuestra la separación de responsabilidades: el controlador no conoce la lógica de combate, el servicio no conoce HTTP ni sesiones, y el modelo no conoce ninguno de los dos.

10. Seguridad

Medida	Implementación	Protege contra
Hash de contraseñas	bcrypt con 10 rondas de sal	Filtración de base de datos
JWT firmado	HS256 con secret en .env	Suplantación de identidad
Variables de entorno	.env excluido de Git (.gitignore)	Exposición de credenciales
Control de roles	Middleware comprueba rol en BD	Escalada de privilegios
Validación de pertenencia	El servicio verifica owner antes de operar	Acceso a recursos ajenos
Sesiones de Express	SESSION_SECRET en .env, cookie httpOnly	Secuestro de sesión
Docker secrets	Credenciales solo en variables de entorno del contenedor	Hardcoding de credenciales

11. Instalación y Configuración

11.1 Requisitos previos

- Docker y Docker Compose
- Node.js v18 o superior (para desarrollo local sin Docker)
- npm 9 o superior

11.2 Pasos de instalación

```
# 1. Clonar e instalar dependencias
git clone <repo-url>
cd pokeback && npm install
```

```
# 2. Configurar variables de entorno
cp .env.example .env
# Editar .env:
# DB_HOST=db DB_PORT=5432 DB_NAME=pokeback DB_USER=... DB_PASSWORD=...
# JWT_SECRET=... SESSION_SECRET=...
# PGADMIN_DEFAULT_EMAIL=... PGADMIN_DEFAULT_PASSWORD=...
```

```
# 3. Arrancar con Docker (inicializa BD automáticamente)
docker compose up
```

```
# Para reinicializar la BD desde cero:
docker compose down -v && docker compose up
```

11.3 Credenciales de prueba

Username	Contraseña	Rol
ash	1234	user
misty	1234	user
test	1234	admin

12. Líneas de Mejora Futuras

Mejora	Descripción	Prioridad
Migraciones versionadas	Sustituir init.sql por sequelize-cli migrations para producción	Alta
Tests automatizados	Suite Jest + Supertest para servicios y endpoints principales	Alta
Rate limiting	Limitar peticiones por IP con express-rate-limit	Alta
Estilo visual PvE	CSS para las vistas battle-select, battle-fight y battle-result	Media
Sistema de niveles	XP por batalla, nivel del Pokémon afecta stats en combate	Media
WebSockets PvP	Combates en tiempo real entre usuarios con Socket.io	Media
Refresh tokens	Añadir refresh token con mayor TTL para mejorar UX	Media
Documentación Swagger	Generar docs interactiva con swagger-jsdoc	Media
Cache de catálogo	Redis para cachear el catálogo de Pokémon (datos estáticos)	Baja

13. Conclusiones

El proyecto POKEback ha cumplido su objetivo principal: demostrar dominio del desarrollo backend con Node.js, Express, Sequelize, PostgreSQL y MongoDB, aplicando buenas prácticas de arquitectura y seguridad en un contexto de trabajo en equipo real, y ha alcanzado un MVP funcional con sistema de combate PvE operativo.

Las decisiones técnicas tomadas: separación de capas MVC con servicios, doble sistema de autenticación, centralización de asociaciones, FK nullable para permadeath, estado de batalla en sesión...han resultado en una base sólida y extensible sobre la que se pueden implementar las futuras mecánicas de juego sin refactorizaciones profundas.

El sistema PvE añade la primera mecánica de juego real al proyecto, cerrando el loop de economía de monedas: los jugadores ganan monedas en batallas para gastarlas en la tienda, y capturan nuevos Pokémon para reforzar su colección, que a su vez les permite afrontar más batallas con mejores stats.

Como aprendizajes clave del equipo destacan: la importancia de definir el modelo de datos y el orden de los routers antes de empezar a codificar, el valor de los middlewares reutilizables para la autenticación, y que una arquitectura bien pensada desde el inicio ahorra tiempo considerable en las fases avanzadas del proyecto.